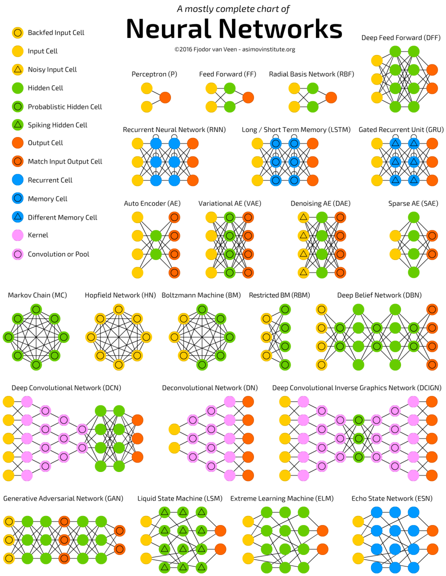
CNNs

UW ECE 657A - Core Topic

Mark Crowley

Lecture Outline

Types of Deep Learning



Types of Deep Neural Networks

- **RBM:** Restricted Boltzman Machines (RBM) - older directed deep model.
- **RNN:** Recurrent Neural Networks (RNN) - allow links from outputs back to inputs, over time, good for time series learning
- **LSTM:** Long-Term Short-Term networks - more complex form of RNN
- **DeepRL:** Deep Reinforcement Learning
- **GAN:** General Adversarial Networks - train two networks at once

Types of Deep Neural Networks

RBM: Restricted Boltzman Machines (RBM) - older directed deep model.

RNN: Recurrent Neural Networks (RNN) - allow links from outputs back to inputs, over time, good for time series learning

LSTM: Long-Term Short-Term networks - more complex form of RNN

- integrate strategically remembering particular information from the past
- formalizes a process for *forgetting* information over time.

DeepRL: Deep Reinforcement Learning

GAN: General Adversarial Networks - train two networks at once

Types of Deep Neural Networks

RBM: Restricted Boltzman Machines (RBM) - older directed deep model.

RNN: Recurrent Neural Networks (RNN) - allow links from outputs back to inputs, over time, good for time series learning

LSTM: Long-Term Short-Term networks - more complex form of RNN

DeepRL: Deep Reinforcement Learning

- CNNs + Fully Connected Deep Network for learning a representation of a policy
- Reinforcement Learning for updating the policy through experience to make improved decision decisions
- Requires a value/reward function

GAN: General Adversarial Networks - train two networks at once

Types of Deep Neural Networks

RBM: Restricted Boltzman Machines (RBM) - older directed deep model.

RNN: Recurrent Neural Networks (RNN) - allow links from outputs back to inputs, over time, good for time series learning

LSTM: Long-Term Short-Term networks - more complex form of RNN

DeepRL: Deep Reinforcement Learning

GAN: General Adversarial Networks - train two networks at once

Lecture Outline

CNN History

After deep networks, and ReLU's, the third major innovation are Convolutional Neural Networks (CNNs).

- Work by Yann LeCun in 80s)
- **LeNet 1** (1993) for digit recognition
- **AlexNet** (2012) for ImageNet challenge - beat competition by error rate of 16% vs 26% for next best
- Cat videos (2012) Quoc Le - the Google cat youtube paper at ICML.
- **ZFNet** (2013), `GoogLeNet` (2014), ...

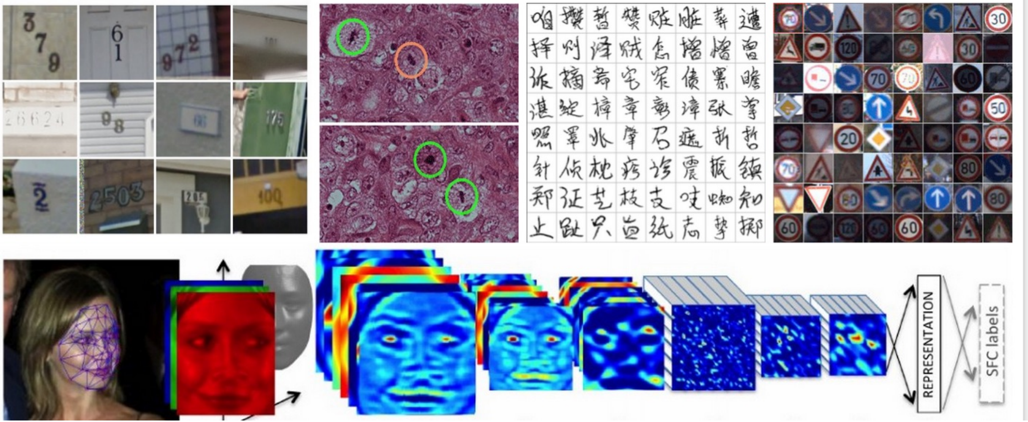
Benefits of CNNs

- **Simple** - repeated operation across each image - essentially a visual filter applied at every location.
    - Related to kernals discussed earlier, but automatically learns the right kernal mapping.

  but we don't define the filters as in older methods and we don't define the kernal, we learn the filter automatically through training.
- **Efficient** - for images compared to a naive shallow, NN approach.
    - Still takes a long time to train, but uses structure of images to reduce duplicated work.

  they still take a long time to train and lots of images, but in a way CNNs train more efficiently on images than general networks do on general data
- **Effective** - Computer vision applications are now mostly using some form of CNNs, but mostly for *low risk tasks*.
- **Natural?** - possible connection to animal visual system

Example Applications of CNNs



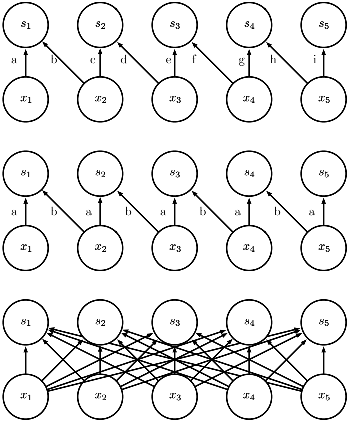
What Makes a Deep Neural Network a CNN?

Essential Property: a neural network that uses convolution in place of general matrix multiplaction in at least one of its layers.

Additional properties:

- Sparse connectivity
- Parameter sharing (tied weights)
- Equivariant Representations

Kinds of Network Connectivity



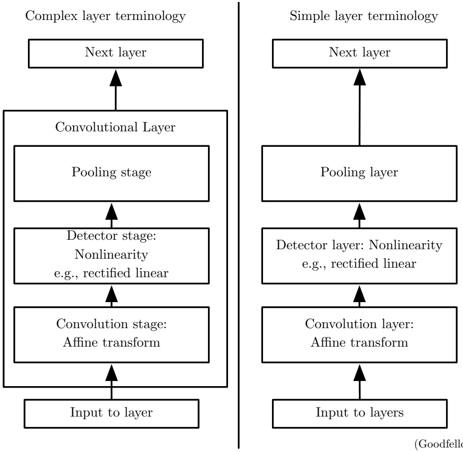
Local connection

Convolution

Fully connected

For convolution we share the paramters

Convolutional Network Structure

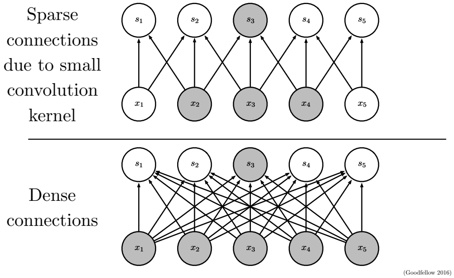


(Goodfellow 2016)

Lecture Outline

Sparse connectivity

- NN: matrix multiplication for parameters mapping all inputs to all outputs.
- CNN: sparse weights, many input pixels (millions), small number of outputs (hundreds, thousands).

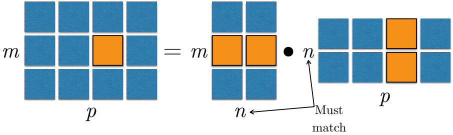


(after) now, how do we do that?

Matrix Dot Product

$$\mathsf{C} = \mathsf{AB}$$

$$C_{i,j} = \sum_k A_{i,k} B_{k,j}$$



(Goodfellow 2016)

Convolution

A filter (kernal/tensor) is defined and passed across all the pixels, providing a response for each pixel that provides a weighted average over nearby pixels.

$$s(t) = \int x(a)w(t-a)da$$

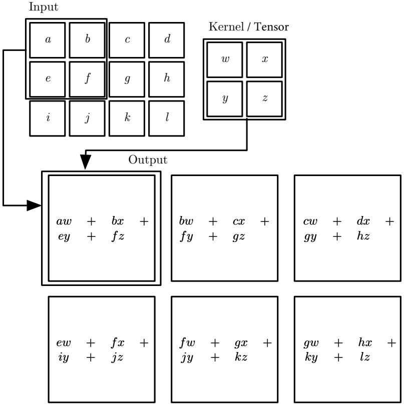
$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a)w(t-a)$$

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(m,n)K(i-m, j-n)$$

where

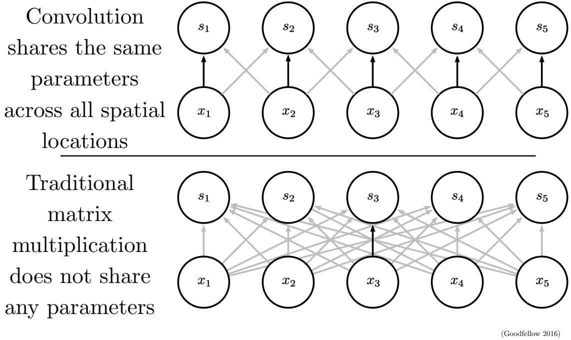
- a - age of measurement in a timeseries
- I - 2D input image
- K - 2D kernal

2D Convolution



(Goodfellow 2016)

Parameter sharing

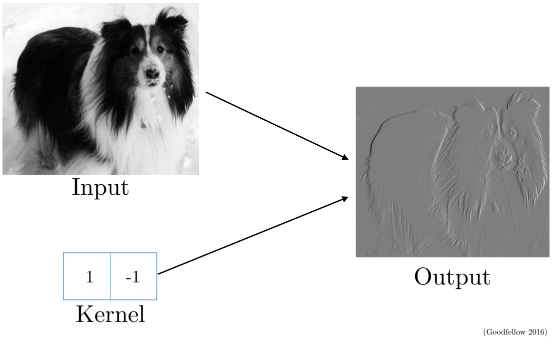


This greatly reduces the number of parameters to train, improves genrlizaabilty too since each weight has to fulfill more rles and commonlities will be focuses into the same weights.

A simple example

(see other slide deck)

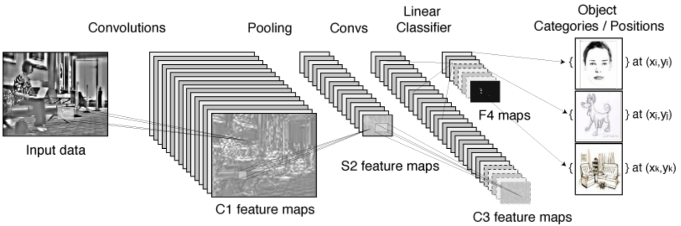
Edge detection by convolution with a kernal that subtracts the value from the neighbouring pixel on the left for every pixel.



Here's a simple example

Convolutional Network Structure

- input data: image (eg. 256x256 pixels x3 channels RGB)



Efficiency of Convolution

Input size: 320 by 280
Kernel size: 2 by 1
Output size: 319 by 280

	Convolution	Dense matrix	Sparse matrix
Stored floats	2	319*280*320*280 $>$ 8e9	2*319*280 = 178,640
Float muls or adds	319*280*3 = 267,960	$>$ 16e9	Same as convolution (267,960)

(Goodfellow 2016)

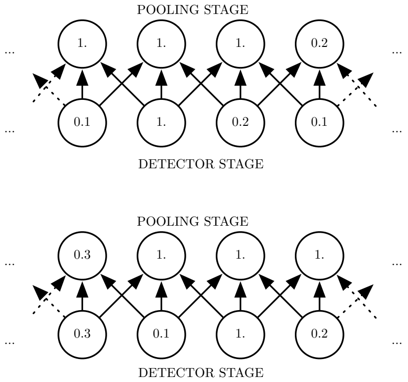
Lecture Outline

Max Pooling

- problem: Still too much density and repetition in the network.
- **observation:** Nearby pixels tend to represent the same thing/class/object.
- solution: Combine, or "pool", responses from nearby nodes.
- Could use `average`, `median`, `minimum`, but `maximum` often works well.

Max Pooling and Invariance to Translation

Pooling refers to aproximating the outputs of a layer by aggregating nearby values. This improves *invariance to small translations* in the input.



(Goodfellow 2016)

Cross-Channel Pooling and Invariance to Learned Transformations

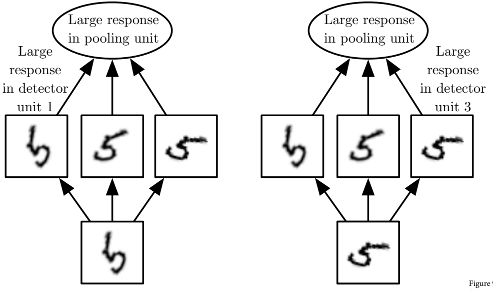
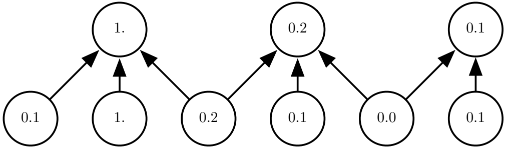


Figure 9.9
(Goodfellow 2016)

Pooling with Downsampling

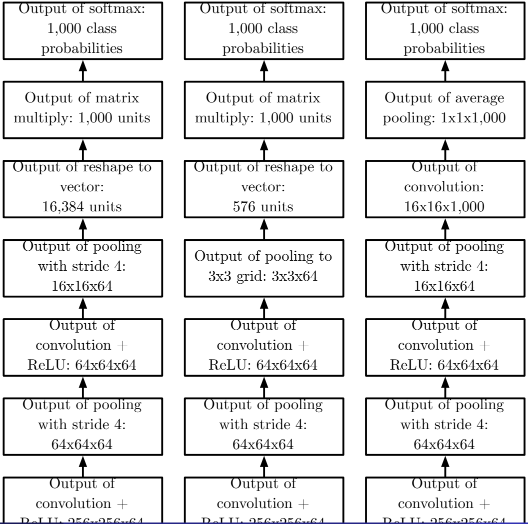
We can also use pooling to reduce the size of each layer to create a hierarchical model.



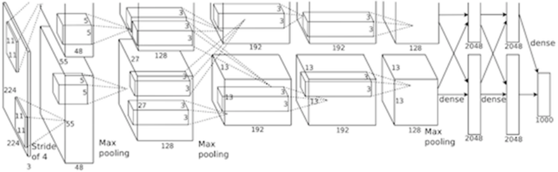
(Goodfellow 2016)

Lecture Outline

CNN Architecture Examples



AlexNet



AlexNet architecture (May look weird because there are two different "streams". This is because the training process was so computationally expensive that they had to split the training onto 2 GPUs)

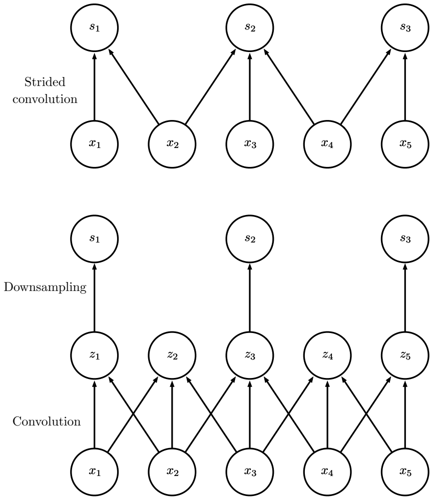
Lecture Outline

Other CNN Modification

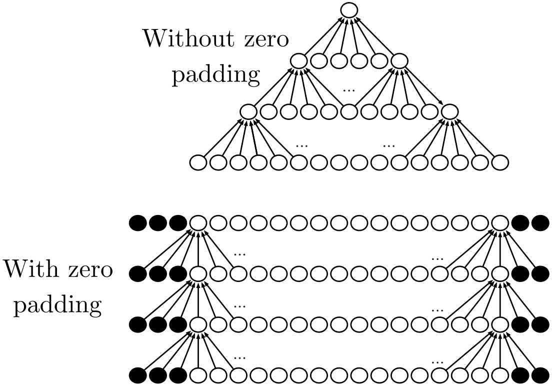
- Strides - number of pixels overlap between adjacent filters
- Zero padding - removing edge pixels from filter scan, can reduce size of network and deal with edge effects
- Alternate local connectivity options
- Partial connectivity between channels

we may even hear of new innovations on this today

Convolutions with Strides

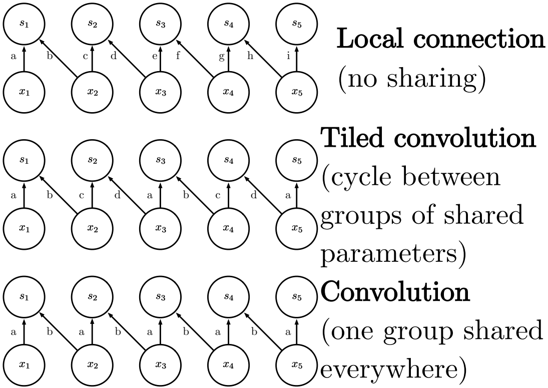


Effect of Zero Padding on Network Size



(Goodfellow 2016)

Tiled Convolution



(Goodfellow 2016)

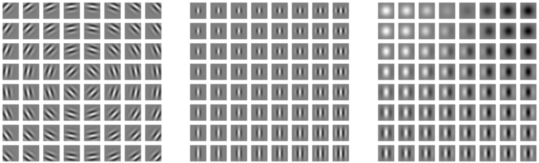
Lecture Outline

CNNs Learn Meaningful Filters

When trained on photographic images the filters learned by CNNs often correspond to prior image processing constructs or to identifiable real world patterns:

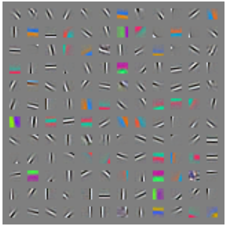
- low level stages learn filters like lines, curves, directional vectors. Similar to Gabor filters from animal visual systems.
- Unsupervised training - top layers can correspond to clusters of images

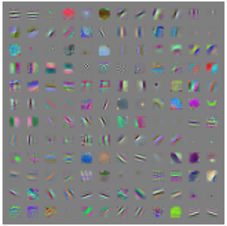
Gabor Functions



(Goodfellow 2016)

Gabor Like Learned Kernals Learned via CNNs





actually many deep learning approaches will learn things similar functions to Gabor functions in the first layer. Edge detectors and directional filters are useful for understanding visual images.

Unsupervised Training of CNNs?

- Cat videos (2012) Quoc Le - the Google cat youtube paper at ICML.
- *Technically,* this was not a CNN at all
  - their approach used the same "local response field" idea as a CNN
  - but they did not convolve it across the image
  - instead, they allowed each part of the field to learn it's own patterns (eye, ear, cute lil'nose....ahem, etc.)
  - they also allows nodes within each layer to communicate with each other, cross connections - more similar to a Deep Belief Net or Restricted Boltzemann Machine.
- More recently on Interpretting CNNs from DeepMind (open): `https://distill.pub/2018/building-blocks/‘9`

Lecture Outline

Major Deep CNN Architectures

keep in mind this changes frequently

Spatial Transducer Net: input size scales with output size, all layers are convolutional

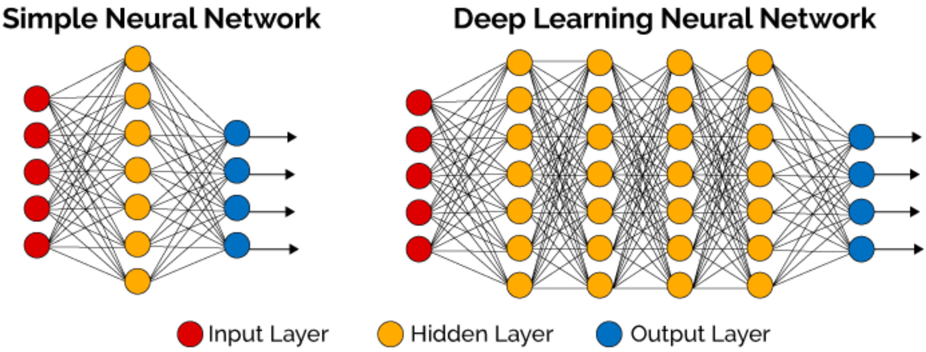
All Convolutional Net: no pooling layers, just use strided convolution to shrink representation size

Inception: complex architecture designed to achieve high accuracy with low computational cost

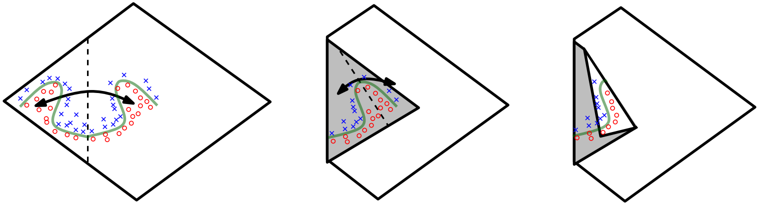
ResNet: blocks of layers with same spatial size, with each layer's output added to the same buffer that is repeatedly updated. Very many updates = very deep net (over 150 layers!) but without vanishing gradient.

Lecture Outline

What Makes Learning Deep?



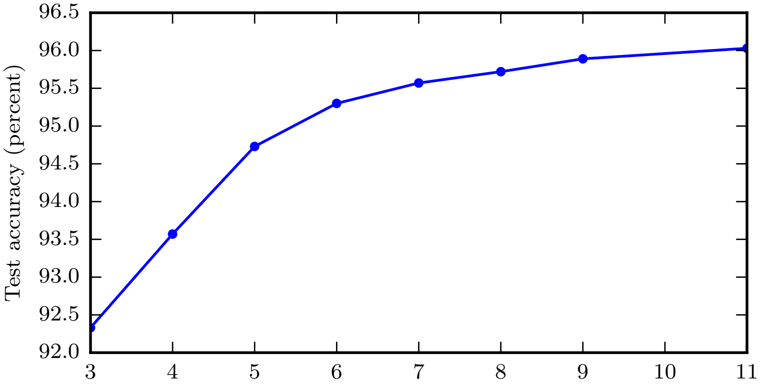
Advantage of Increased Depth



(Goodfellow 2016)

Using Hidden LeRU units, each hidden layer increases power, exponential advantage of additional layers.

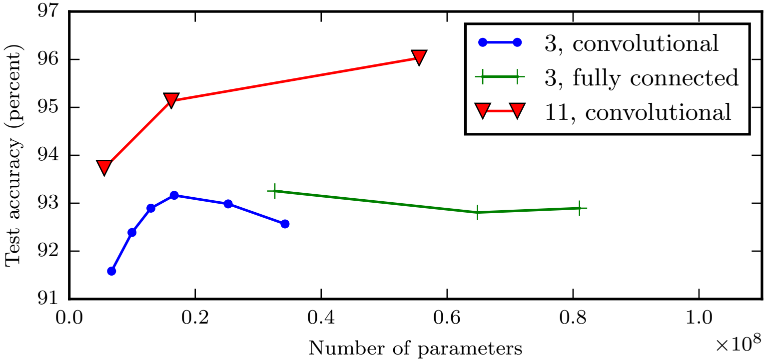
Better Generalization with Greater Depth



(Goodfellow 2016)

Increasing accuracy from number of layers in detection of numbers in photos of adresses.

Large, Shallow Models Overfit More



(Goodfellow 2016)